

Введение в SQL

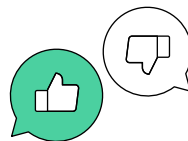


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Юрий Новиков

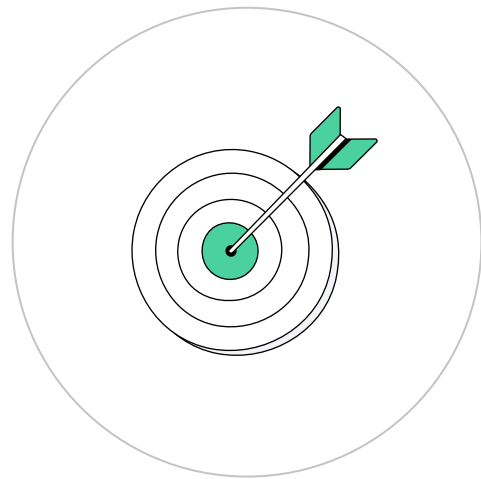
О спикере:

- В IT с 2000 года.
- В настоящее время ведущий разработчик в ОМК-ИТ, ранее занимал руководящие должности.
- Принимал участие в разработке веб-приложений и прикладного ПО. Одна из систем - система мониторинга городского пассажирского транспорта.
- Программирую: Delphi, C#, Php, JavaScript.
- Проектирует базы, взаимодействию с базами посредством СУБД MySql, MSQl, Oracle.
- Проводит курсы по реляционным базам данных.



Сегодня вы узнаете

- Что такое язык запросов SQL
- Из каких компонентов состоит SQL
- Основные операторы языка SQL

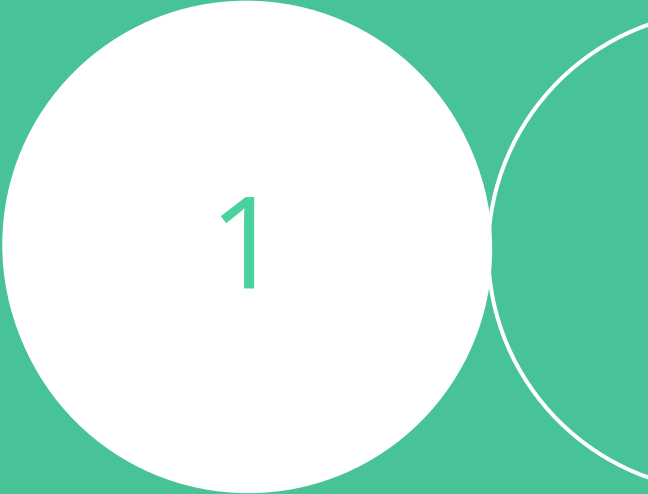


План занятия

- 1 SQL
- 2 DDL
- 3 DML
- 4 DCL
- 5 TCL
- 6 Итоги



SQL



1

SQL

SQL — это язык запросов для реляционных баз данных.

SQL — отраслевой стандарт, поэтому многие даже не реляционные СУБД поддерживают SQL (MongoDB, DynamoDB, ClickHouse, Cassandra и др.)

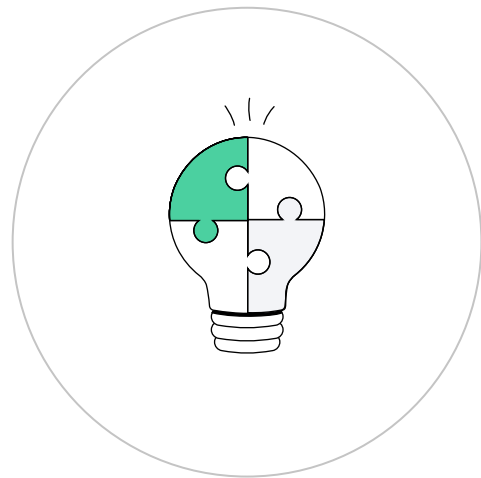
Современный SQL позволяет не только взаимодействовать с данными, но и практически полностью конфигурировать и контролировать все компоненты СУБД. Что делает SQL необходимым как для задач аналитики, так и для задач администрирования.

Диалект SQL может незначительно отличаться для каждой конкретной СУБД

Логический порядок

SQL, как любой язык программирования, имеет свою структуру и синтаксис.

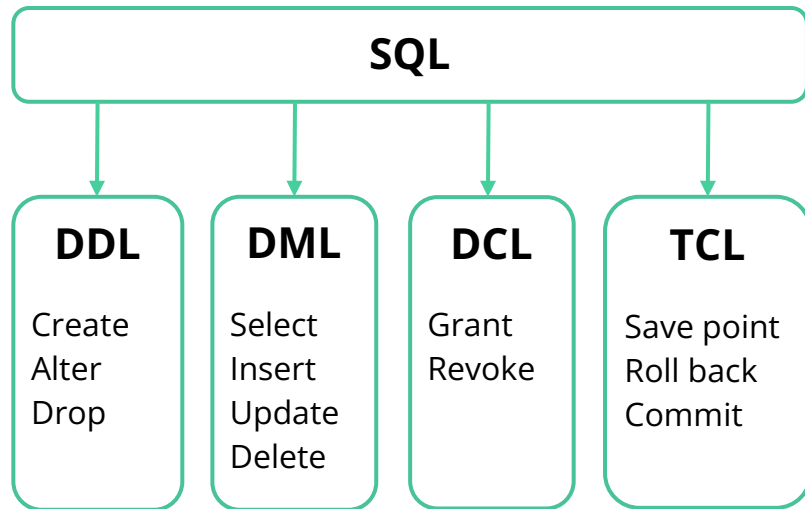
Прежде чем перейти к работе с этим языком, нужно запомнить, что структура запросов имеет свой порядок, но под капотом интерпретатор будет выполнять запрос в определённой логической последовательности, и от этого будут зависеть области видимости и с какими данными что будет происходить



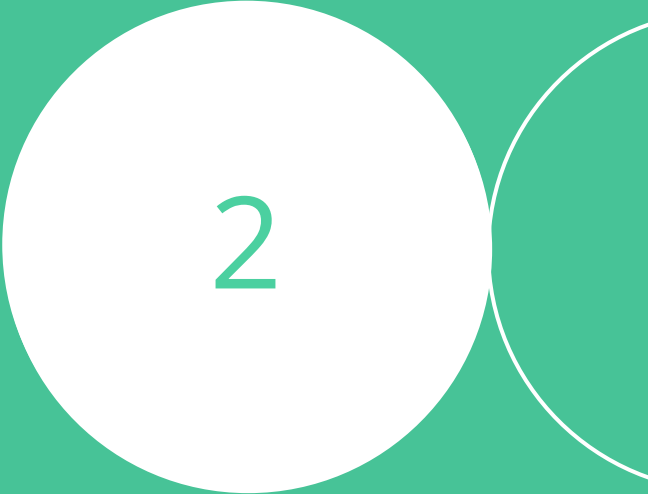
DDL, DML, DCL, TCL

SQL-запросы делятся на четыре группы:

- **DDL** — описывает структуры данных
- **DML** — работает с самими данными
- **DCL** — управляет доступом
- **TCL** — управляет транзакциями



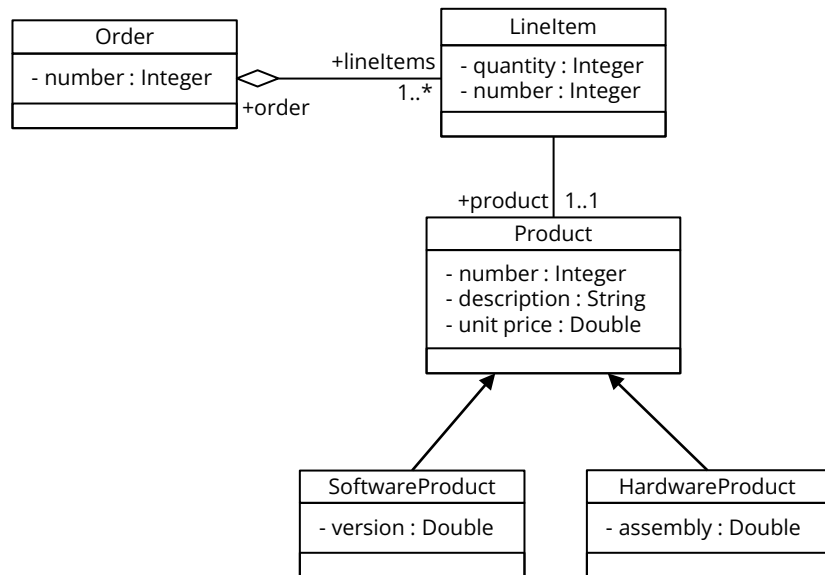
DDL



2

DDL

Data Definition Language (DDL) — это группа операторов определения структуры данных. С помощью этих операторов определяются таблицы и схема их взаимодействия



DDL

В DDL входят следующие операторы:

- **CREATE** — используется для создания объектов базы данных
- **ALTER** — используется для изменения объектов базы данных
- **DROP** — используется для удаления объектов базы данных

DDL. CREATE

Создание базы данных:

```
CREATE DATABASE `dvd-rental`;
```

Создание таблицы:

```
CREATE TABLE customer(id SERIAL PRIMARY KEY, fio CHAR(50) UNIQUE, dob DATE);
```

DDL. CREATE

Создание представления:

```
CREATE VIEW underage AS SELECT fio FROM customer WHERE TIMESTAMPDIFF(YEAR, dob, CURDATE()) >= 18;
```

Создание индекса:

```
CREATE UNIQUE INDEX dob_idx ON customer(dob);
```

DDL. ALTER

Изменение имени базы данных:

```
ALTER DATABASE old_name MODIFY NAME = new_name;
```

* синтаксис T-SQL, не работает в MySQL

Изменение имени таблицы:

```
ALTER TABLE customer RENAME TO persons;
```

DDL. ALTER

Изменение столбцов в таблице:

```
ALTER TABLE persons ADD password CHAR(20) NOT NULL;
```

Изменение типа данных атрибута:

```
ALTER TABLE persons MODIFY fio VARCHAR(100);
```


DDL. DROP

Удалить базу данных:

```
DROP DATABASE `dvd-rental`;
```

Удалить таблицу:

```
DROP TABLE persons;
```

DDL. DROP

Удалить индекс:

```
DROP INDEX dob_idx ON persons;
```

Удалить представление:

```
DROP VIEW underage;
```

DML



3

DML

Data Manipulation Language (DML) — это группа операторов для манипуляции данными. С помощью этих операторов можно добавлять, изменять, удалять и получать данные из базы данных

№ группы	Код модуля	Дата проведения	Эксперт
1	SLINB	2023-06-27	Иван
2	BASH	2024-05-18	Сергей
3	NET	2025-08-19	Анна
4	SDBSQL	2026-04-09	Владимир
5	SYSDB	2027-09-12	Олег
6	SFLT	2028-03-04	Юлия

DML

В DML входят самые распространённые операторы:

- **SELECT** — осуществляет выборку данных
- **INSERT** — добавляет новые данные
- **UPDATE** — изменяет существующие данные
- **DELETE** — удаляет данные

DML. SELECT

Вывести все записи таблицы:

```
SELECT * FROM persons;
```

Вывести количество записей в таблице:

```
SELECT COUNT(*) FROM persons;
```

DML. SELECT

Вывести определённые столбцы из таблицы:

```
SELECT fio, dob FROM persons;
```

Вывести данные по условию:

```
SELECT fio, dob FROM persons WHERE id > 25;
```

Логический порядок обработки инструкции SELECT

1. FROM
2. ON
3. JOIN
4. WHERE
5. GROUP BY
6. WITH CUBE или WITH ROLLUP
7. HAVING
8. SELECT
9. OVER
10. DISTINCT
11. ORDER BY

DML. INSERT

Вставка данных в таблицу:

```
INSERT INTO persons VALUES (6, «Иванов Иван», «1985-07-12»);
```

Вставка данных в таблицу с указанием столбца:

```
INSERT INTO persons (fio) VALUES («Петров Петр»);
```

Вставка данных в таблицу из другой таблицы:

```
INSERT INTO persons (fio) SELECT fio FROM customers;
```

DML. UPDATE

Изменение значения в конкретной строке:

```
UPDATE persons SET fio = 'Максим Сергеевич' WHERE id = 1;
```

Изменение значений по всему атрибуту:

```
UPDATE persons SET fio = 'Максим Сергеевич';
```

Важно: если не указывать конкретную строку через оператор WHERE — изменения произойдут во всех строках таблицы

DML. DELETE

Удалить все данные в таблице:

```
DELETE FROM persons;
```

Удалить конкретную строку или набор строк:

```
DELETE FROM persons WHERE id IN (1, 3, 7);
```

Важно: если не указывать конкретную строку через оператор WHERE — будут удалены все данные из таблицы

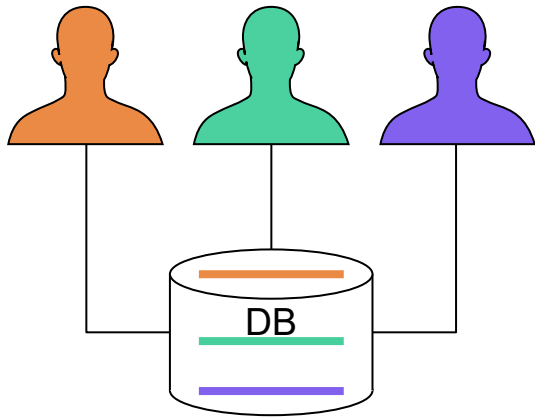
DCL



4

DCL

Data Control Language (DCL) — группа операторов определения доступа к данным. Эти операторы нужны для управления разрешениями доступа к данным и выполнения операций над объектами базы данных. Права назначаются на пользователя или на роли



DCL

В DCL входят следующие операторы:

- **GRANT** — предоставляет пользователю или группе разрешения на определённые операции с объектами
- **REVOKE** — отзывает выданные разрешения
- **DENY** — задаёт запрет, имеющий приоритет над разрешением, отсутствует в MySQL

DCL. GRANT

Дать все права пользователю к базе данных и, если его не существует, то создать его до версии 8.0:

```
GRANT ALL PRIVILEGES ON *.* TO 'dbuser'@'localhost' IDENTIFIED BY 'password' WITH GRANT OPTION;
```

Создать пользователя и дать все права к базе данных после версии 8.0:

```
CREATE USER 'dbuser'@'localhost' IDENTIFIED BY 'password';  
GRANT ALL PRIVILEGES ON *.* TO 'dbuser'@'localhost';
```

Вставка данных в таблицу из другой таблицы:

```
GRANT SELECT, UPDATE ON `dvd-rental`.* TO 'dbuser'@'localhost'
```

DCL. REVOKE

Убрать все права пользователя к базе данных:

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'dbuser'@'localhost';
```

Удалить пользователя после того, как убрали права:

```
DROP USER 'dbuser'@'localhost';
```

Убрать определённые права у пользователя:

```
REVOKE INSERT, DELETE ON `dvd-rental`.* FROM 'dbuser'@'localhost'
```


DCL. DENY

В MySQL отсутствует оператор DENY, все права назначаются через GRANT и удаляются через REVOKE.

На примере других СУБД синтаксис будет выглядеть следующим образом:

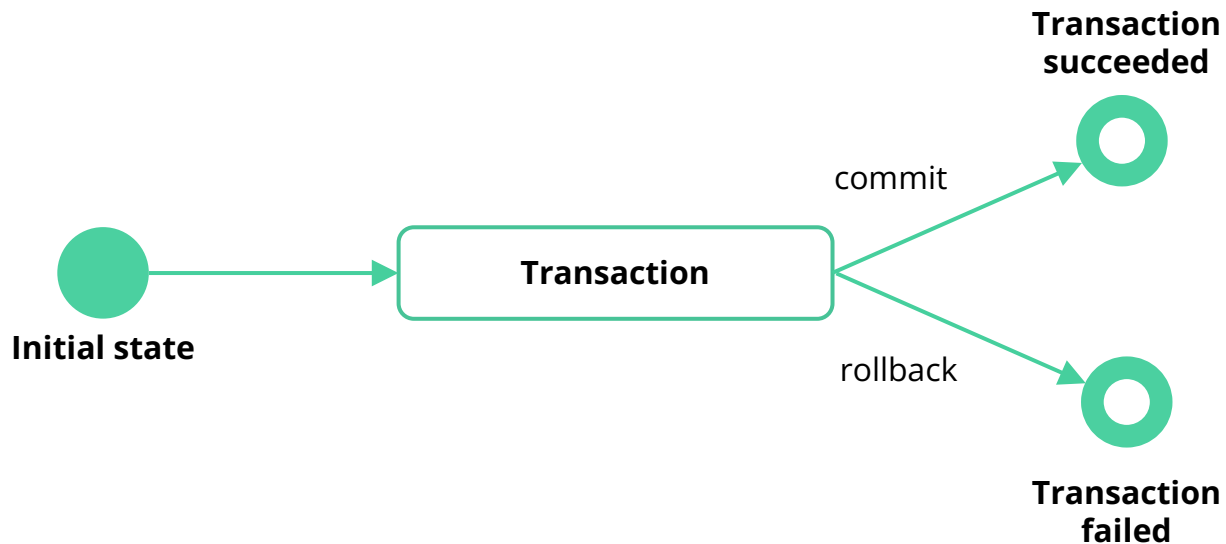
```
DENY privilege_name ON object_name TO {user_name | public |  
    role_name};
```

TCL

5

TCL

Transaction Control Language (TCL) — группа операторов для управления транзакциями



TCL

TCL состоит из следующих операторов:

- **START TRANSACTION** — служит для определения начала транзакции
- **COMMIT** — применяет транзакцию
- **ROLLBACK** — откатывает все изменения, сделанные в контексте текущей транзакции
- **SAVEPOINT** — устанавливает промежуточную точку сохранения внутри транзакции

TCL

Пример транзакции для MySQL:

```
# начало транзакции
START TRANSACTION;
# обновляем данные
UPDATE accounts SET balance = balance - 1000.00 WHERE id = 10;
# ставим точку сохранения
SAVEPOINT my_savepoint;
# обновляем данные
UPDATE accounts SET balance = balance + 1000.00 WHERE id = 27;
# допустили ошибку, возвращаемся к my_savepoint
ROLLBACK TO my_savepoint;
# теперь правильно обновляем данные
UPDATE accounts SET balance = balance + 1000.00 WHERE id = 37;
# завершаем транзакцию
COMMIT;
```

TCL

Пример транзакции для PostgreSQL:

-- 1. Начало транзакции

BEGIN TRANSACTION; -- Для MySQL: **START TRANSACTION;**

-- 2. Проверка баланса (опционально, для безопасности)

```
DECLARE @current_balance DECIMAL(10,2);
```

```
SELECT @current balance = balance FROM accounts WHERE account id = 1;
```

-- 3. Основные операции

UPDATE accounts SET balance = balance - 100.00 -- Списание

WHERE account_id = 1 **AND** balance >= 100.00; -- Условие предотвращения отрицательного
 баланса

UPDATE accounts SET balance = balance + 100.00 -- Зачисление

WHERE account id = 2;

TCL

Пример транзакции для PostgreSQL:

-- 4. Фиксация или откат

IF @@ROWCOUNT = 2 AND @current_balance >= 100.00 -- Проверка успешности обоих обновлений

COMMIT; -- Подтверждаем изменения

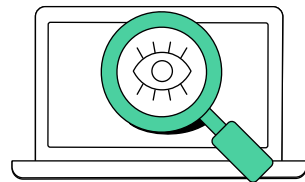
ELSE

ROLLBACK; -- Отменяем все операции

RAISERROR('Transaction failed: insufficient funds or account error', 16, 1);

Демонстрация работы

Посмотрим, как работать с SQL-запросами



Итоги



6

Итоги занятия

В лекции мы:

- узнали, что такое SQL
- узнали про логический порядок выполнения запросов
- познакомились с DDL, DML, DCL и TCL запросами



Введение в SQL

